TransferIa · Documentos de cambios

Primeros planos y video

Entrega 11 · Caras inclinadas, primeros planos, bordes, video de prueba y audio desfasado

Sin commit: carpeta de trabajo al 27 de octubre de 2026

Comparado con: d6b94d6

Documento generado: 27 de octubre de 2026

Pruebas: caras generadas por IA, en CPU

Contenido

1. Resumen de la entrega 2
2. Caras inclinadas: ahora se puede apagar 3
3. Caras en primer plano 3
3.1 El problema 3
3.2 La solución: alejar la cámara sin alejarla 4
4. Sin borde oscuro 5
4.1 El problema 5
4.2 La solución 6
5. Oclusión en zoom extremo 6
6. Video de prueba 7
6.1 Para qué 7
6.2 Cómo usarlo 7
6.3 Probado: cuánto tiempo ahorra 9
7. Videos con el audio desfasado 9
7.1 El problema 9
7.2 La solución 10
8. Cómo se comprobó 11
9. Lo que aprendimos 11
Glosario 11

Si el índice aparece vacío, en Word haz clic derecho sobre él → Actualizar campos.

1. Resumen de la entrega

> **Nota:** Estos cambios **todavía no tienen commit**: están en la carpeta de trabajo de `D:\TransferIa` al 27 de octubre. Se probaron con una copia de esos archivos y, para el "antes", con el commit `d6b94d6`. Todas las pruebas usan caras **generadas por IA**; los números marcados "(README)" son mediciones de la sesión de desarrollo escritas en el README que no se volvieron a medir.

Cambio	Interfaz	CLI	Qué gana el usuario
Caras inclinadas se puede apagar	Casilla **Caras inclinadas**	`--sin-inclinadas`	Volver a la detección original si hace falta comparar
Caras en primer plano	—	—	Selfies y primeros planos: antes la cara no se encontraba
Sin borde oscuro	—	—	La cara cerca del borde ya no deja una franja negra
Oclusión en zoom extremo	—	—	La máscara ya no recorta piel por error
Video de prueba	Botón **Video de prueba**	`--prueba [FPS]`	Ver cómo queda en un tercio del tiempo
Videos desfasados	Aviso en la barra de estado	Aviso en la consola	El audio ya no queda corrido

2. Caras inclinadas: ahora se puede apagar

La detección de caras inclinadas y boca abajo (entrega 3) estaba siempre activa. Ahora es una casilla, **Caras inclinadas**, en la fila **Caras**, marcada por defecto. En la CLI se apaga con `--sin-inclinadas`.

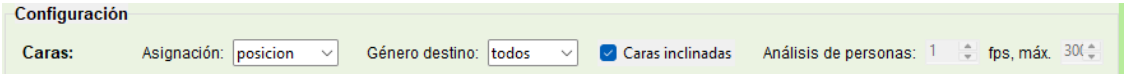


Figura 1. *La casilla nueva en la fila "Caras"*

Sirve para comparar o para volver al comportamiento original de insightface. Con un destino girado 90°, la detección original encuentra la cara pero ubica mal ojos, nariz y boca, y el resultado sale deformado:

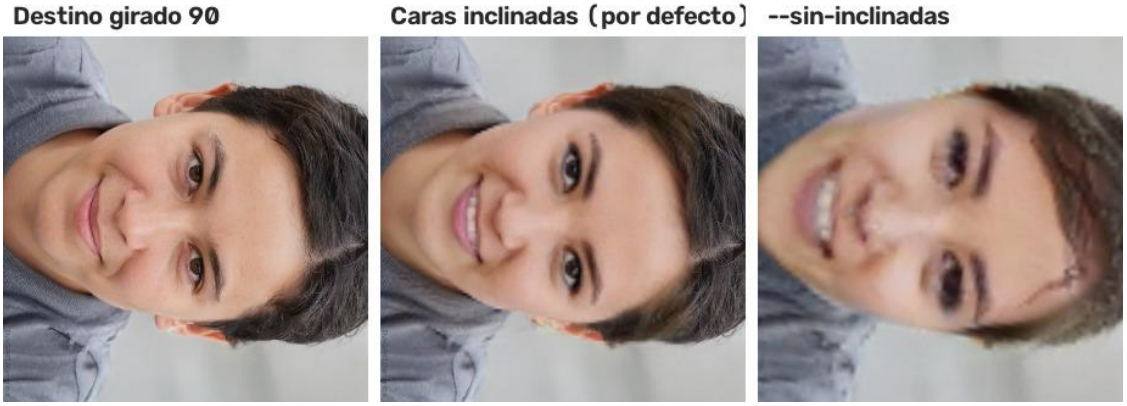


Figura 2. *La misma cara girada 90°, con la opción activada (al medio) y con --sin-inclinadas (derecha)*

Medido como diferencia media con el destino: **5,7** con la opción activada y **17,4** con `--sin-inclinadas` (la cara deformada cambia mucho más de lo que debería).

Otro ajuste: la cara ya no "tiembla" cerca de los 30°

Antes, a partir de 30° de inclinación los puntos de la cara se recalculaban (con la imagen enderezada) y por debajo no. Si la cabeza oscilaba cerca de los 30°, los puntos saltaban de un método al otro entre frames. Ahora, entre **10° y 20°**, se mezclan de a poco los dos resultados:

```
peso = clip((abs(angulo) - REFINAR_DESDE) / (REFINAR_COMPLETO - REFINAR_DESDE), 0, 1)   # 10° -> 0, 20° -> 1
mejor.kps = (1 - peso) * cara.kps + peso * mejor.kps      # mitad y mitad a los 15°
```

Según el README, el mayor salto entre un grado y el siguiente bajó de **4,2 %** a **0,7 %** de la distancia entre ojos (README). Como los puntos cambian, un video empezado antes de este cambio no se puede continuar: la "firma" incluye ahora `deteccion: 2`.

3. Caras en primer plano

3.1 El problema

El detector de caras (RetinaFace, dentro de buffalo_l) está entrenado con caras que ocupan una parte del cuadro. Cuando la cara **llena casi todo** (selfie, primer plano), no la encuentra.

Cómo se midió

1. Con cada una de las 5 caras de IA se armaron 7 recortes de 512×512 en los que la cara ocupa el 40, 55, 70, 80, 100, 120 y 140 % del ancho. Se guardó dónde están de verdad sus 5 puntos (ojos, nariz, comisuras).
2. Con cada versión se detectó como al procesar (detector a 480 px) y como al crear modelos (a 640 px).

3. Error = distancia media de los puntos detectados a los reales, en % de la distancia entre ojos.

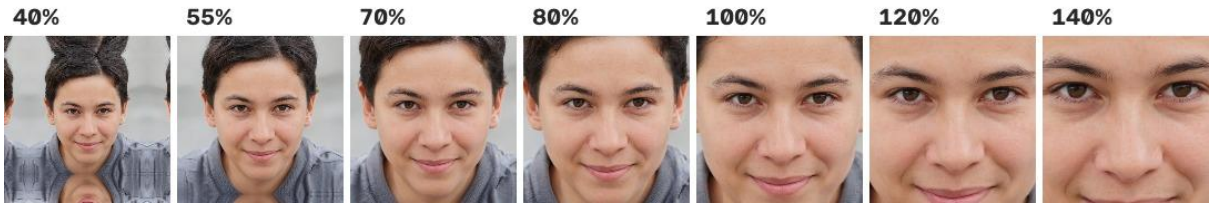


Figura 3. *Una de las caras de prueba, del 40 % al 140 % del ancho del cuadro*

La cara ocupa	Antes, al procesar	Antes, al crear modelos	Ahora, al procesar	Ahora, al crear modelos
40 %	5 de 5	5 de 5	5 de 5	5 de 5
55 %	5 de 5	3 de 5 (error hasta 19 %)	5 de 5	5 de 5
70 %	4 de 5 (error hasta 18 %)	0 de 5	5 de 5	5 de 5
80 %	2 de 5 (error 17-29 %)	0 de 5	5 de 5	5 de 5
100 %	0 de 5	0 de 5	5 de 5 (error 2-4 %)	5 de 5 (3-5 %)
120 %	0 de 5	0 de 5	5 de 5 (3-5 %)	5 de 5 (2-5 %)
140 %	0 de 5	0 de 5	5 de 5 (3-7 %)	5 de 5 (3-8 %)

Qué hay que mirar: antes, desde el 70-80 % la cara se perdía o se encontraba con los puntos muy corridos (un error de 20 % es como poner el ojo a un quinto de la distancia entre ojos de su lugar). Ahora se encuentra siempre, con un error de 1 a 8 %.

3.2 La solución: alejar la cámara sin alejarla

Si el detector no ve una cara porque está demasiado cerca, se le muestra la imagen **más chica dentro de un marco**: la cara pasa a ocupar la mitad o un tercio del cuadro, el tamaño que el detector conoce. Después se restan los píxeles del marco a las coordenadas, para volver a la imagen original.

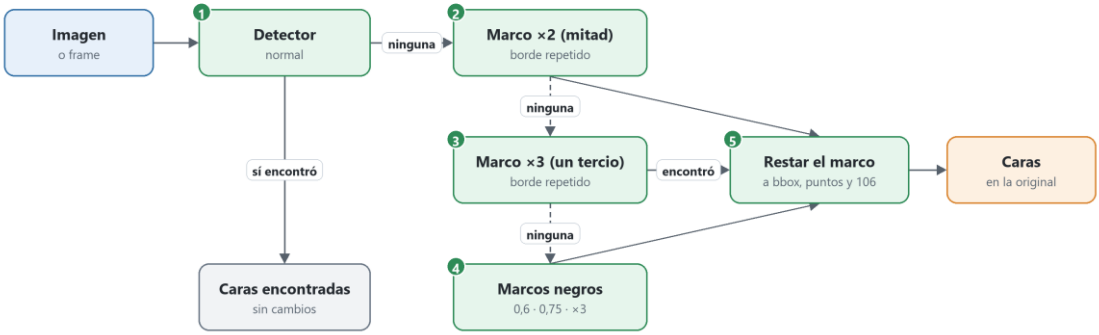


Figura 4. *Orden de los intentos en _detectar_primer_plano() (core/Face_Analyzer.py)*

1. Detecta normal. Si encuentra alguna cara, termina: **las imágenes que ya funcionaban no cambian**.
2. Si no, prueba con un marco que repite el borde de la imagen (`BORDER_REPLICATE`), primero del 50 % de cada lado (imagen a la mitad) y después del 100 % (a un tercio).
3. Si tampoco, prueba con marcos **negros** de 0,6, 0,75 y 1,0. Con el negro la cara se encuentra en zooms extremos, aunque con los puntos algo menos precisos: por eso va al final.

4. A la cara encontrada se le restan los píxeles del marco (recuadro, 5 puntos, 106 puntos y 68 puntos 3D).

```
BORDES_PRIMER_PLANO = ((0.5, cv2.BORDER_REPLICATE), (1.0, cv2.BORDER_REPLICATE), (0.6, cv2.BORDER_CONSTANT),
                       (0.75, cv2.BORDER_CONSTANT), (1.0, cv2.BORDER_CONSTANT))
...
for factor, borde in BORDES_PRIMER_PLANO:
    p = int(max(alto, ancho) * factor)
    con_marco = cv2.copyMakeBorder(frame, p, p, p, p, borde, value=(0, 0, 0))
    caras = Analyzer_Faces._ANALIZADOR_CARA.get(con_marco)
    if caras:
        for c in caras:
            self._desplazar_cara(c, -p, -p)     # del frame con marco al original
        return caras
```

> **Nota:** Funciona siempre, con o sin **Caras inclinadas**, y también al crear modelos. El costo: solo cuando no hay ninguna cara se hacen hasta 5 detecciones más por imagen.

> **Para aprender:** Cuando un modelo falla con entradas fuera de su rango, a veces basta con **transformar la entrada** para llevarla a ese rango (achicar, girar, enmarcar) y deshacer la transformación en la salida. Lo mismo se hizo con las caras boca abajo en la entrega 3.

4. Sin borde oscuro

4.1 El problema

inswapper trabaja con un recorte cuadrado y alineado de la cara (un poco más grande que la cara). En un primer plano, o con la cara pegada al borde, ese recorte **se sale de la imagen**, y lo que queda afuera se rellenaba con negro. Al pegar la cara de vuelta, ese negro entraba en el resultado como una franja oscura en el borde.



Figura 5. *Cara al 70 % del ancho. Abajo, la esquina superior izquierda aumentada*

Qué hay que mirar: en "Antes" el borde de arriba y de la izquierda tiene una línea oscura y la imagen se ve más borrosa en la esquina. Medido en las 5 caras, en una franja de 12 px alrededor del borde: lo que

cambió quedaba entre un **11 y un 16 % más oscuro** que el original; ahora la diferencia va de 0 a +11 % (es la piel de la otra cara, no un oscurecimiento). Una de las 5 caras (la 3) la versión anterior ni siquiera la detectó.

4.2 La solución

Antes de pegar, `_margen_recorte` calcula cuántos píxeles se sale el recorte. Si se sale, se trabaja sobre una copia del frame **con un margen de borde repetido** (no negro), se pega la cara ahí y al final se recorta el margen.

```
margen = self._margen_recorte(frame.shape[:2], cara_dst, modelo.input_size[0])
if margen:
    grande = cv2.copyMakeBorder(frame, margen, margen, margen, margen, cv2.BORDER_REPLICATE)
    resultado = self.pegar_cara(grande, cara_src, self._cara_desplazada(cara_dst, margen), ...)
    return resultado[margen:margen + alto, margen:margen + ancho].copy()
```

Si la cara entra entera (`margen == 0`), no se hace nada distinto: según el README, en ese caso el resultado es idéntico píxel a píxel al anterior (README).

5. Oclusión en zoom extremo

La máscara de oclusión (XSeg) marca qué parte de la cara es piel visible. En un zoom extremo, el recorte alineado también se sale de la imagen y se rellena **repitiendo el borde**: eso crea rayas que el modelo confunde con un objeto delante de la cara, y **recorta piel** que no está tapada.

La solución: si parte del recorte cae afuera, se calcula la máscara dos veces más, con dos rellenos sin rayas (**espejo** y **difuso**), y se toma el mayor. Un falso objeto que sale de un relleno no sale en el otro; un objeto real está dentro del cuadro y lo marcan los dos.

```
valido = cv2.warpAffine(np.full((h, w), 255, np.uint8), M, (256, 256), borderValue=0) > 127
if not valido.all():                                    # parte del recorte cae fuera del cuadro
    espejo = cv2.warpAffine(img, M, (256, 256), borderMode=cv2.BORDER_REFLECT_101, ...)
    difuso = crop.copy()
    difuso[~valido] = cv2.GaussianBlur(crop, (0, 0), 12)[~valido]
    mask = np.maximum(self._inferir(espejo), self._inferir(difuso))
```

Cómo se comprobó

Se compararon las dos máscaras (la de `d6b94d6` y la nueva) sobre los 25 primeros planos del 70 al 140 %. La vieja quitaba de la piel: **0,4-2,8 %** al 70-80 %, **1,3-15 %** al 100 % y **6-21 %** al 120-140 %.

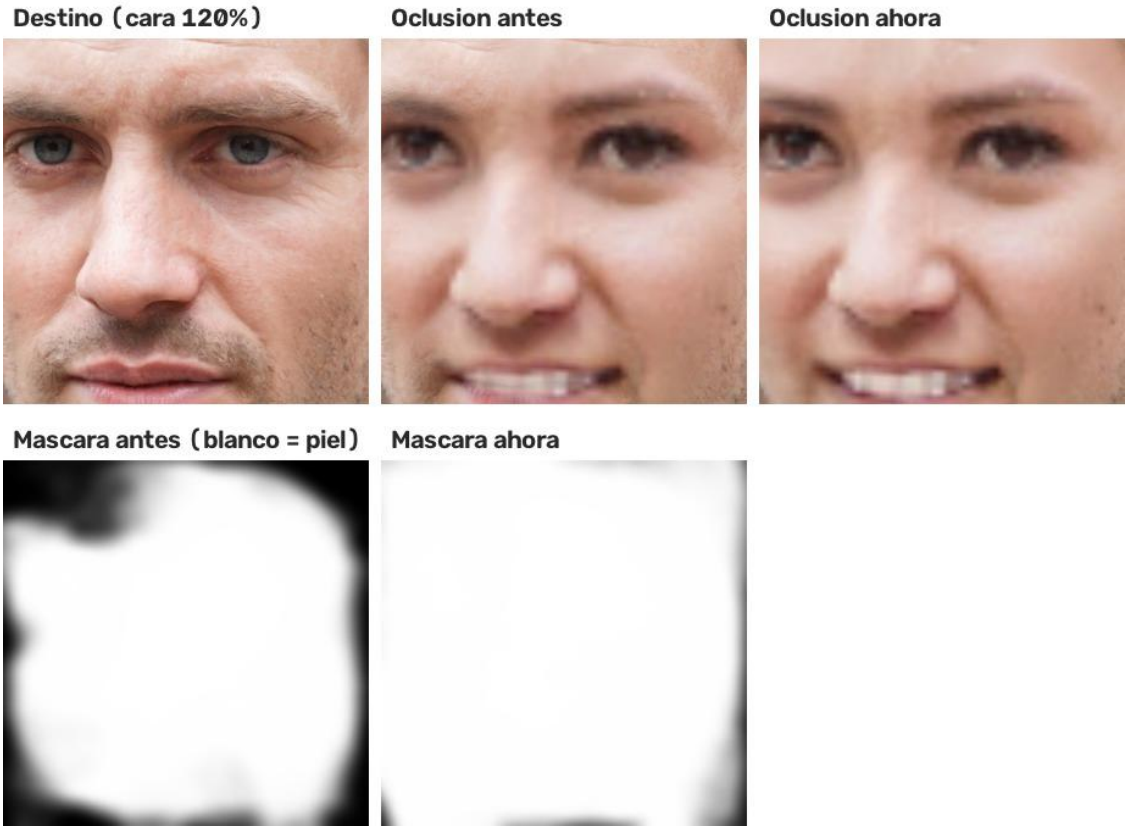


Figura 6. *Cara al 120 % con oclusión: arriba los resultados, abajo las máscaras (blanco = piel que se cambia)*

Qué hay que mirar: la máscara de antes tiene manchas negras arriba a la izquierda y abajo (piel que dejaba sin cambiar); la nueva cubre toda la cara. En el resultado, en esta prueba, la diferencia es pequeña: la piel que la máscara vieja dejaba fuera queda casi en el borde de la imagen.

6. Video de prueba

6.1 Para qué

Procesar un video entero para descubrir que una opción no convenía hace perder mucho tiempo. El **video de prueba** procesa solo un tramo y con menos frames por segundo, con las opciones actuales, para juzgar parecido, color, oclusión y ojos antes del proceso largo.

6.2 Cómo usarlo

1. Elige la fuente y el video destino, y ajusta las opciones como para el proceso final.
2. Opcional: marca **Solo un fragmento** y elige el tramo. Sin fragmento se usan **los primeros 10 s**.
3. En la fila nueva, elige los fps (10 por defecto, de 1 a 30) y pulsa **Video de prueba**.
4. Al terminar ofrece abrirlo. Se guarda como `<nombre>_prueba.mp4` en la carpeta de salida; el resultado final no se toca.

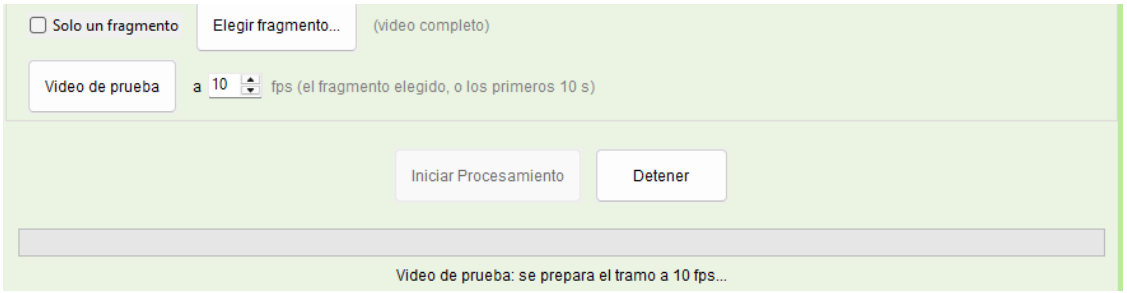


Figura 7. *La fila "Video de prueba" y la barra de estado mientras prepara el tramo*

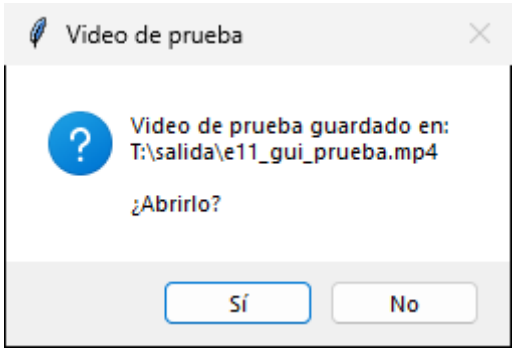


Figura 8. *Al terminar*

En la CLI es `--prueba` (10 fps) o `--prueba FPS`, y se combina con `--start` y `--end`:

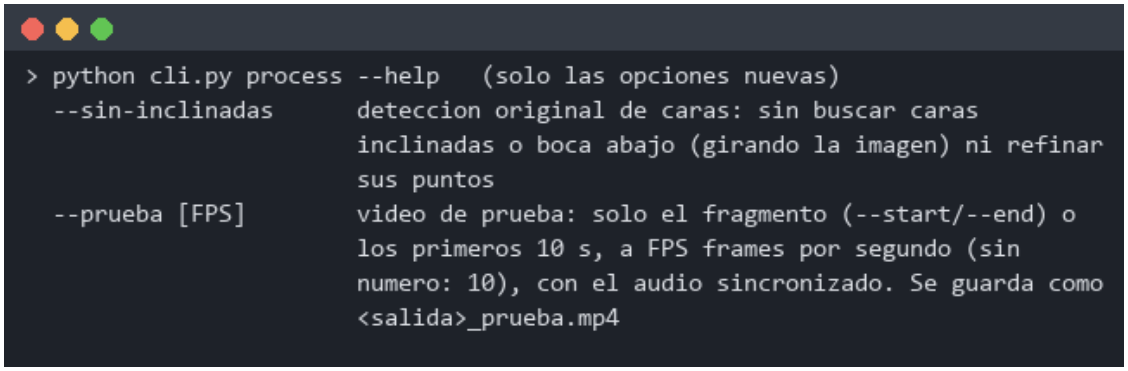


Figura 9. *Las dos opciones nuevas en la ayuda*

```
> python cli.py process --source fuente.jpg --target video_20s.mp4 --output salida/e11_video.mp4
   --prueba
Fragmento: 0.00s a 10.00s (10.00s, 300 frames)
Video de prueba: se prepara el tramo a 10 fps...
Procesando: [----------------------------]   1.00% (1/100)
   ...
Procesando: [############################] 100.00% (100/100)
Resultado guardado en: <prueba>\salida\e11_video_prueba.mp4

> ffprobe salida/e11_video_prueba.mp4  (fps, frames, duracion)
r_frame_rate=10/1
nb_read_frames=100
duration=10.000000

> python cli.py process --source fuente.jpg --target video_20s.mp4 --output salida/e11_frag.mp4
  --prueba 5 --start 0:12 --end 0:16
Fragmento: 12.00s a 16.00s (4.00s, 120 frames)
Video de prueba: se prepara el tramo a 5 fps...
Procesando: [#---------------------------]   5.00% (1/20)
   ...
Procesando: [############################] 100.00% (20/20)
Resultado guardado en: <prueba>\salida\e11_frag_prueba.mp4

> ffprobe salida/e11_frag_prueba.mp4
r_frame_rate=5/1
nb_read_frames=20
duration=4.000000
```

Figura 10. *Los primeros 10 s a 10 fps, y del segundo 12 al 16 a 5 fps, con ffprobe*

6.3 Probado: cuánto tiempo ahorra

Mismo tramo de 10 s (video de 30 fps)	Frames	Tiempo (CPU)
Proceso normal (`--end 10`)	300	490 s
Video de prueba a 10 fps (`--prueba`)	100	172 s (35 %)

¿Por qué 10 fps? Es un tercio del trabajo y alcanza para juzgar la cara. Con menos se pierden parpadeos y movimientos de boca, y cada imagen puede quedar hasta medio frame corrida respecto del sonido: 50 ms a 10 fps.

Sincronía comprobada: en un video con dos "claps" (pantalla blanca de 0,3 s + pitido, a los 3 y 7 s), el video de prueba a 10 fps tiene el destello y el pitido en el mismo momento (desfase **0 ms**).

> **Nota:** En el primer intento los destellos eran de un solo frame (1/30 s) y en el video a 10 fps **no aparecieron**: dos de cada tres frames se descartan. Es esperable y es justo lo que se pierde con pocos fps (parpadeos).

7. Videos con el audio desfasado

7.1 El problema

Algunos videos (grabaciones de VLC, de pantalla o del celular) tienen **ritmo de frames variable**, o la imagen y el audio **no empiezan en el mismo momento**. Un reproductor lo compensa con los tiempos de cada frame. Transferla, en cambio, escribía los frames a ritmo fijo desde 0 y pegaba el audio desde 0: el resultado quedaba desfasado.

Cómo se reprodujo

1. Se armó un video de 10 s en el que la imagen empieza **0,47 s** después que el audio (como algunas grabaciones), con dos claps a los 3 y 7 s. Al reproducirlo, destello y pitido van juntos (desfase 3 ms).
2. Se procesó con la versión anterior y con la nueva.
3. Un script (`medir_desfase.py`) busca los frames blancos (con sus tiempos reales) y los pitidos en el audio, y resta.

```
> version anterior (d6b94d6): python cli.py process --source fuente.jpg --target video_desfasado.mkv
   --output salida/e11_desfase_antes.mp4
Procesando: [------------------------------]   0.32% (1/316)
   ...
Procesando: [############################--]  94.94% (300/316)
Resultado guardado en: <prueba>\salida\e11_desfase_antes.mp4

> version nueva: mismo comando
Video con el audio y la imagen empiezan con 0.47 s de diferencia: se prepara una copia a ritmo fijo
  para que el audio no quede desfasado...
Procesando: [------------------------------]   0.32% (1/314)
   ...
Procesando: [##############################] 100.00% (314/314)
Resultado guardado en: <prueba>\salida\e11_desfase_ahora.mp4

> python medir_desfase.py datos/video_desfasado.mkv salida/e11_desfase_antes.mp4 salida/e11_desfase_
  ahora.mp4
video_desfasado.mkv: destellos [3.467, 7.467] pitidos [3.47, 7.47] desfase (ms) [3, 3]
e11_desfase_antes.mp4: destellos [3.0, 7.0] pitidos [3.47, 7.47] desfase (ms) [470, 470]
e11_desfase_ahora.mp4: destellos [3.467, 7.467] pitidos [3.47, 7.47] desfase (ms) [3, 3]
```

Figura 11. *Antes: el sonido llega 470 ms tarde. Ahora: 3 ms, igual que el original*

Qué hay que mirar: además del desfase, la versión anterior creía que había **316** frames (por la duración total del archivo) y la barra terminó en 300/316 (94,94 %).

7.2 La solución

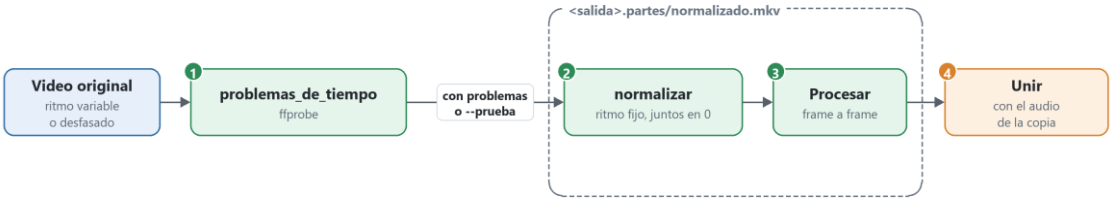


Figura 12. *Cuando hay problemas de tiempo (o es un video de prueba) se trabaja sobre una copia normalizada*

1. `problemas_de_tiempo()` pregunta a **ffprobe** el ritmo nominal y el promedio del video (si difieren más de 1 %, es variable) y el inicio de la imagen y del audio (si difieren más de 20 ms, están corridos).
2. Si hay problemas, avisa y `normalizar()` hace con **ffmpeg** una copia del tramo a ritmo fijo, con imagen y audio empezando juntos en 0 (si a la imagen le faltan frames al principio repite el primero; si al audio le falta, agrega silencio). El audio va sin pérdida (PCM).
3. Se procesa esa copia frame a frame, como siempre.
4. Al unir, el audio sale **de la copia**, que ya coincide con la imagen. La copia se borra al terminar.

```
comando += ["-map", "0:v:0", "-vf", f"fps={fps:.6f}:start_time=0", "-c:v", "libx264", "-crf", "12", ...]
if tiene_audio:
    comando += ["-map", "0:a:0", "-af", "aresample=async=1:first_pts=0", "-c:a", "pcm_s16le"]
```

`fps=...:start_time=0` reparte los frames a ritmo fijo desde el segundo 0 según su tiempo real; `aresample=async=1:first_pts=0` estira o rellena el audio para que también empiece en 0. El **video de prueba** usa la misma función, con los fps elegidos.

> **Nota:** El mensaje de aviso tiene un error de redacción: dice "Video con el audio y la imagen empiezan con 0.47 s de diferencia". Es solo el texto (une dos frases); no afecta el resultado.

> **Para aprender:** Un video es una lista de imágenes **con su momento**, no solo una lista de imágenes. Al procesarlo hay que respetar esos momentos o fijarlos antes (normalizar).

8. Cómo se comprobó

- Destino girado 90° con y sin `--sin-inclinadas`.
- 35 primeros planos (5 caras × 7 tamaños) detectados con la versión anterior y la nueva, a 480 y 640 px, con el error de los puntos.
- 5 caras al 70 % procesadas con las dos versiones; brillo medido en el borde.
- Máscara XSeg vieja y nueva en 25 primeros planos; una cara al 120 % procesada con `--occlusion`.
- Video de prueba de 10 s y de un fragmento, contado con ffprobe; tiempos contra el proceso normal; sincronía con claps.
- Video con la imagen corrida 0,47 s, procesado con las dos versiones y medido con `medir_desfase.py`.
- En la ventana: casilla **Caras inclinadas**, botón **Video de prueba** y su mensaje final.

9. Lo que aprendimos

- Cuando un detector falla fuera de su rango, se puede llevar la entrada a su rango y deshacer el cambio en la salida.
- Rellenar con negro lo que cae fuera de la imagen deja rastros; repetir o reflejar el borde no.
- Dos versiones del mismo cálculo con rellenos distintos separan lo real (sale en ambas) de lo inventado (sale en una).
- Un corte brusco entre dos métodos produce saltos; una mezcla gradual los elimina.
- Un "ensayo" barato (video de prueba) ahorra procesos largos fallidos.

Glosario

Término	Significado
Primer plano	Toma en la que la cara ocupa casi todo el cuadro.
BORDER_REPLICATE	Relleno que repite el último píxel del borde hacia afuera.
BORDER_REFLECT_101	Relleno que refleja la imagen como un espejo.
Ritmo de frames variable (VFR)	Video en el que el tiempo entre un frame y el siguiente no es siempre el mismo.
PCM	Audio sin comprimir y sin pérdida.
Clap	Señal de sincronía: algo que se ve y se oye en el mismo instante.